Machine Coding: Debounced Search

- **Problem:** Build a search box that calls a product search API (`dummyjson.com/products/search?q=`) only after the user stops typing for 400 ms, instead of on every keystroke. Show "Searching...", results, "no results", an error with Retry, and a hint for too-short queries. Old responses must never overwrite newer ones, and repeated queries should come from a cache.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Assumptions (delay, min length) written down
Use cases + architecture + state	7 min	Table, hook split, two-query idea
Search without debounce, then `useDebounce`	12 min	Requests only after a pause
Loading / error / empty / min length	10 min	All UI states
Race conditions (AbortController) + cache	12 min	Correct under fast typing
Test manually (Network tab) + wrap-up	14 min	Demo request count + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I keep two values: `query` updates on every keystroke so the input stays responsive, and `debouncedQuery` only updates after 400 ms of quiet. The API only ever sees the debounced one."* Open the Network tab while you demo, so they can *see* one request instead of five.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Text input; the search runs automatically after the user pauses typing (debounce, 400 ms).
- Ignore empty / too-short input (< 2 characters after trimming) and show a hint.
- Show "Searching..." while waiting or loading, then the results (title, category, price, thumbnail).
- "No products found for 'xyz'" when the API returns nothing.
- An error message with Retry when the request fails.
- A clear (×) button.
- A response for an old query must never replace the results of the current one.
- A query that was already searched comes from a cache (no new request).

Non-functional requirements

Concern	What it means here
Performance	Typing "iphone" fast sends 1 request instead of 6. The input itself is never delayed: only the API call is.
Scalability	Less load on the server; the cache removes repeated requests; the request can move to server-side throttling later without UI changes.
Error handling	`response.ok` check (fetch doesn't throw on 404/500), response shape check, `AbortError` ignored, Retry.
Accessibility	Labelled input, `role="search"`, status messages in `aria-live="polite"`, error in `role="alert"`, `aria-busy` on stale results, clear button with `aria-label`.
Reusability	`useDebounce(value, delay)` works for any value; `useProductSearch(query)` hides the fetching, cancelling and caching.
Extensibility	Swap the API in `api/products.js`; add an autocomplete dropdown, highlighting or pagination on top of the same hooks.

Clarifying questions to ask the interviewer

1. What delay: 300 ms, 500 ms? Should Enter search immediately?
2. Minimum characters before searching? What about only spaces?
3. Should older results stay visible while the new search loads, or should the list clear?
4. Is there a real API (which one, any rate limits?) or should I mock it?
5. Should results be cached? For how long (data can change)?
6. Is this a results page or an autocomplete dropdown (keyboard navigation, select an item)?

Assumptions we will build with

- 400 ms debounce, minimum 2 characters after trim, no Enter shortcut.
- Real API: `GET https://dummyjson.com/products/search?q=…&limit=10`.
- Old results stay visible but faded while the next search is pending (less flicker).
- In-memory cache per query for the session (key = trimmed, lowercased query).
- A results list, not a dropdown (keyboard navigation is an extension).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type "p", "ph", "pho", "phon", "phone" quickly	Input updates instantly; "Searching..."; **one** request for "phone" after 400 ms
Pause while typing ("pho" … 500 ms … "phone")	Two requests ("pho", then "phone"); only the latest results are shown
Type "a"	"Type at least 2 characters."; no request
Type " " (spaces)	Treated as empty; no request
Request in flight, user types a new query	Old request aborted; its response never shows
API returns products	"10 results for 'phone'" + list
API returns `[]`	`No products found for "xyzq".`
API 404/500 or network down	`Search failed (HTTP 404).` + Retry
Click Retry	Same query requested again
Search "phone", then "laptop", then "phone" again	Third search comes from the cache: request count doesn't grow
Type "c++" or "50% off"	Query URL-encoded (`c%2B%2B`), so the request is correct
Click ×	Input cleared; results and status cleared
Type "phone", delete back to "ph" within 400 ms	Only "ph" is searched (the timer restarts on every change)

3. Design the architecture

Module	Responsibility
`App`	Owns `query` ; derives the debounced/valid search term and UI flags; renders input, status, results
`hooks/useDebounce.js`	`useDebounce(value, delay)` : returns the value after it stops changing for `delay` ms
`hooks/useProductSearch.js`	Given a search term: fetch, cancel the stale request, cache, expose `{ status, results, error, requestCount, retry }`
`api/products.js`	`searchProducts(query, { signal })` : build the URL, check `ok` and the response shape
`ResultItem`	Shows one product

```
App  (query)
 │  debouncedQuery = useDebounce(query, 400)
 │  searchTerm     = valid(debouncedQuery) ? trimmed : ""
 │  { status, results, error, retry } = useProductSearch(searchTerm)
 ├── input + clear button
 ├── status line (aria-live): hint / Searching... / N results / no results
 ├── error + Retry
```

```
 └── <ul> results.map → ResultItem
useProductSearch → api/products.js → fetch(dummyjson)
```

Start simple. First version: `useEffect` on `query` that fetches directly (show the request flood in the Network tab), then add `useDebounce`, then move the fetching into `useProductSearch` once you add cancelling + caching.

4. Design the state / data model

State	Where	Purpose
`query`	App	Raw input value, updates every keystroke (controlled input)
`debounced`	useDebounce	The value after the pause
`status`	useProductSearch	`idle | loading | success | error`
`results`	useProductSearch	Products for the last successful term
`error`	useProductSearch	Error message
`attempt`	useProductSearch	Retry counter (changing it re-runs the effect)
`requestCount`	useProductSearch	Demo only: proves the debounce works
`cache` (ref)	useProductSearch	`Map<term, products>`; a ref because changing it shouldn't re-render

```
const [query, setQuery] = useState("");
const debouncedQuery = useDebounce(query, DEBOUNCE_MS);

const term = debouncedQuery.trim();
const searchTerm = term.length >= MIN_LENGTH ? term : "";
const { status, results, error, retry } = useProductSearch(searchTerm);

// derived UI flags
const typed = query.trim();
const isTooShort = typed.length > 0 && typed.length < MIN_LENGTH;
const isWaiting = typed !== term && typed.length >= MIN_LENGTH; // timer still running
const isBusy = isWaiting || status === "loading";
```

Senior note

- **Two values, one source:** `query` is the only stored input; `debouncedQuery` is a delayed copy of it, and `searchTerm`, `isTooShort`, `isWaiting` and `isBusy` are derived. There's no separate `isTyping` state to keep in sync.
- **Don't debounce the input itself.** `onChange` must update `query` immediately, or typing feels laggy and characters can get lost. Debounce only the *effect* (the request).
- **Ownership:** `App` owns what the user typed; `useProductSearch` owns everything about the network (status, results, cache). The cache is a `ref`: it is data, not UI.

5. Design the data structures and contracts

API response (trimmed with `select=`):

```
{
  "products": [
    { "id": 121, "title": "iPhone 5s", "category": "smartphones", "price": 199.99, "thumbnail":
"https://…/thumbnail.webp" }
  ],
  "total": 23, "skip": 0, "limit": 10
}
```

`id` is stable and is used as the React key.

API contract

Call	Input	Success	Failure
`searchProducts(query, { signal })` → `GET /products/search?q={encoded}&limit=10&select=title,category,price,thumbnail`	trimmed query, `AbortSignal`	`Product[]`	throws `Error("Search failed (HTTP 404).")`, `Error("Unexpected response…")`, or `AbortError` (ignored)

Hook contracts

```
useDebounce(value, delay) → debouncedValue
useProductSearch(term)    → { status, results, error, requestCount, retry }
// term "" → status "idle", no request
```

Cache key: `term.trim().toLowerCase()`, so "Phone" and "phone " share one entry.

6. Trace the data flow

User types "phone" quickly:

```
Keystrokes p → ph → pho → phon → phone (each < 400 ms apart)
  ↓ onChange → setQuery(...) each time → input re-renders instantly
  ↓ useDebounce effect: each change clears the previous timer and starts a new 400 ms timer
  ↓ isWaiting = true → "Searching..." (aria-live)
  ↓ 400 ms after the last key: setDebounced("phone")
  ↓ searchTerm = "phone" (≥ 2 chars) → useProductSearch effect runs
  ↓ cache miss → AbortController created → status "loading" → fetch(?q=phone)
  ↓ response ok → cache.set("phone", products) → status "success"
  ↓ UI: "10 results for "phone"" + list; request count = 1
(if the user types again before the response: the effect cleanup aborts this fetch)
```

7. Important technical challenges

Challenge 1: debouncing correctly in React

- *Problem:* A `debounce()` created inside the component body is re-created on every render, so each keystroke gets a *new* timer and nothing is debounced. Wrapping it in `useCallback(debounce(fn), [])` freezes `fn` with stale state (and needs an eslint-disable).
- *Solution:* Debounce the **value** with a hook: an effect that sets a timer and clears it on the next change.

```
export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}
```

Challenge 2: race conditions (out-of-order responses)

- *Problem:* The user searches "ph" (slow response, 2 s), then "phone" (fast, 200 ms). "phone" results show, then the late "ph" response arrives and **overwrites** them. Debounce alone doesn't fix this: it reduces requests but doesn't order them.
- *Solution:* One `AbortController` per request; the effect cleanup aborts the previous one, and `AbortError` is ignored.

```
useEffect(() => {
  const controller = new AbortController();
  searchProducts(key, { signal: controller.signal })
    .then(setResults…)
    .catch((err) => { if (err.name === "AbortError") return; setError… });
  return () => controller.abort();
}, [query, attempt]);
```

Challenge 3: UI states without flicker

- *Problem:* Clearing the list on every keystroke makes the page jump; showing "No results" while the timer is still running is wrong (we haven't searched yet); a 1-letter query fires a useless, huge request.
- *Solution:* Derive the flags from `query` vs `debouncedQuery` + `status`. Keep old results visible but faded (`aria-busy`) while busy, and gate requests on a minimum length.

```
const isWaiting = typed !== term && typed.length >= MIN_LENGTH;
const isBusy = isWaiting || status === "loading";
{!isBusy && status === "success" && results.length === 0 && <p>No products found…</p>}
<ul className={isBusy ? "results stale" : "results"} aria-busy={isBusy}>…</ul>
```

Challenge 4: caching and safe requests

- *Problem:* "phone" → "laptop" → "phone" sends the same request twice. `?q=c++` without encoding sends `q=c ` (a `+` means a space in a URL). `fetch` resolves on 404, so `data.products.map` crashes on an error body.
- *Solution:* A `Map` in a ref keyed by the normalised term, `encodeURIComponent`, and `ok` + shape checks.

```
if (cache.current.has(key)) { setState({ status: "success", results: cache.current.get(key), error: "" });
return; }
const url = `${API_URL}?q=${encodeURIComponent(query)}&limit=10`;
if (!response.ok) throw new Error(`Search failed (HTTP ${response.status}).`);
if (!Array.isArray(data.products)) throw new Error("Unexpected response from the server.");
```

8. Implement in the correct order

#	Step	Working result
1	Controlled input + `searchProducts` + `useEffect` on `query` (no debounce)	Works, but 1 request per keystroke (show the Network tab)
2	`useDebounce(query, 400)` ; effect depends on the debounced value	1 request after the pause
3	Render results with `key={p.id}`	Results visible
4	Trim + minimum length 2 + hint	No useless requests
5	`status` enum: loading / success / error; `ok` + shape check	Async states
6	"No results" + "N results" messages	Empty state
7	Move the fetching into `useProductSearch`	Clean `App`
8	`AbortController` cleanup; ignore `AbortError`	No race conditions
9	Retry (`attempt` counter)	Recoverable errors
10	Cache `Map` in a ref	Repeated queries are free
11	`isWaiting` / `isBusy` , faded stale results, clear button	Smooth UX
12	`aria-live` , `role="alert"` , labels; request counter for the demo	Accessible + provable

9. Cover all states and edge cases

State / edge case	Handling
Initial (idle)	Empty input, no status, no request
Typing (timer running)	"Searching...", old results faded
Loading (request in flight)	"Searching...", `aria-busy` on the list
Success	"N results for 'x'" + list
Empty result	`No products found for "x".`
Error	Message with the HTTP status + Retry (`role="alert"`)
1 character	"Type at least 2 characters."; no request
Only spaces	Trimmed to empty → idle
Mixed case / trailing space	Same cache key; same results

State / edge case	Handling
Special characters (`c++` , `&` , `%`)	`encodeURIComponent`
Fast typing	1 request (debounce)
Slow typing with pauses	Several requests; stale ones aborted
Out-of-order responses	Impossible: the previous request is aborted
Same query again	Served from the cache
Clear (×)	Back to idle; any in-flight request aborted
Unmount during a request	Cleanup aborts the request and clears the timer
API shape changes / HTML error page	Shape check → error state instead of a crash

10. Final structure

```
_05_debouncing_search/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                ← global reset
    ├── App.jsx                  ← query state, derived flags, UI
    ├── App.css
    ├── api/
    │   └── products.js          ← searchProducts (encode, ok + shape checks)
    ├── hooks/
    │   ├── useDebounce.js       ← debounce a value
    │   └── useProductSearch.js  ← fetch + abort + cache + retry
    └── components/
        └── ResultItem.jsx       ← one product row
```

11. Full working code

`src/api/products.js`

```
const API_URL = "https://dummyjson.com/products/search";

// shortcut: ?fail in the page URL makes the first request fail, to demo the error state.
let failNext = new URLSearchParams(window.location.search).has("fail");

// GET /products/search?q=phone&limit=10 -> { products: [{ id, title, category, price, thumbnail }], total }
export async function searchProducts(query, { signal } = {}) {
  const url = `${failNext ? API_URL + "-broken" : API_URL}?
q=${encodeURIComponent(query)}&limit=10&select=title,category,price,thumbnail`;
  failNext = false;
  const response = await fetch(url, { signal });
  if (!response.ok) throw new Error(`Search failed (HTTP ${response.status}).`);
  const data = await response.json();
```

```
  if (!Array.isArray(data.products)) throw new Error("Unexpected response from the server.");
  return data.products;
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from "react";

// Returns `value` only after it has stopped changing for `delay` ms.
export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer); // every keystroke cancels the previous timer
  }, [value, delay]);

  return debounced;
}
```

src/hooks/useProductSearch.js

```
import { useEffect, useRef, useState } from "react";
import { searchProducts } from "../api/products";

const IDLE = { status: "idle", results: [], error: "" };

// Fetches results for an (already debounced) query. Cancels stale requests and caches by query.
export function useProductSearch(query) {
  const [state, setState] = useState(IDLE); // status: idle | loading | success | error
  const [requestCount, setRequestCount] = useState(0);
  const [attempt, setAttempt] = useState(0);
  const cache = useRef(new Map());

  useEffect(() => {
    const key = query.trim().toLowerCase();
    if (!key) {
      setState(IDLE);
      return;
    }
    if (cache.current.has(key)) {
      setState({ status: "success", results: cache.current.get(key), error: "" });
      return;
    }

    const controller = new AbortController();
    setState((prev) => ({ ...prev, status: "loading", error: "" })); // keep old results visible while
loading
```

```
    setRequestCount((n) => n + 1);
    searchProducts(key, { signal: controller.signal })
      .then((results) => {
        cache.current.set(key, results);
        setState({ status: "success", results, error: "" });
      })
      .catch((err) => {
        if (err.name === "AbortError") return; // a newer query replaced this one
        setState({ status: "error", results: [], error: err.message });
      });

    return () => controller.abort();
  }, [query, attempt]);

  return { ...state, requestCount, retry: () => setAttempt((a) => a + 1) };
}
```

src/App.jsx

```
import { useState } from "react";
import "./App.css";
import { useDebounce } from "./hooks/useDebounce";
import { useProductSearch } from "./hooks/useProductSearch";
import ResultItem from "./components/ResultItem";

const DEBOUNCE_MS = 400;
const MIN_LENGTH = 2;

function App() {
  const [query, setQuery] = useState(""); // what the user types: updates on every keystroke
  const debouncedQuery = useDebounce(query, DEBOUNCE_MS); // what we search for: updates after a pause

  const term = debouncedQuery.trim();
  const searchTerm = term.length >= MIN_LENGTH ? term : "";
  const { status, results, error, requestCount, retry } = useProductSearch(searchTerm);

  const typed = query.trim();
  const isTooShort = typed.length > 0 && typed.length < MIN_LENGTH;
  const isWaiting = typed !== term && typed.length >= MIN_LENGTH; // debounce timer still running
  const isBusy = isWaiting || status === "loading";

  return (
    <main className="container">
      <h1>Debounced Product Search</h1>

      <div className="search-box" role="search">
        <label htmlFor="search" className="sr-only">
          Search products
        </label>
```

```
        <input
          id="search"
          type="search"
          placeholder="Search products, e.g. phone"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          autoComplete="off"
          autoFocus
        />
        {query && (
          <button type="button" onClick={() => setQuery("")} aria-label="Clear search">
            ×
          </button>
        )}
      </div>

      <p className="meta">API requests sent: {requestCount}</p>

      <div aria-live="polite" className="status">
        {isTooShort && <p>Type at least {MIN_LENGTH} characters.</p>}
        {!isTooShort && isBusy && <p>Searching...</p>}
        {!isBusy && status === "success" && results.length === 0 && <p>No products found for "{term}".</p>}
        {!isBusy && status === "success" && results.length > 0 && <p>{results.length} results for "{term}"
</p>}
      </div>

      {status === "error" && !isWaiting && (
        <div role="alert" className="error">
          <p>{error}</p>
          <button type="button" onClick={retry}>
            Retry
          </button>
        </div>
      )}

      {results.length > 0 && searchTerm && (
        <ul className={`results ${isBusy ? "stale" : ""}`} aria-busy={isBusy}>
          {results.map((p) => (
            <ResultItem key={p.id} product={p} />
          ))}
        </ul>
      )}
    </main>
  );
}

export default App;
```

src/components/ResultItem.jsx

```
const ResultItem = ({ product }) => (
  <li className="result">
    <img src={product.thumbnail} alt="" width="48" height="48" loading="lazy" />
    <div>
      <p className="result-title">{product.title}</p>
      <p className="result-meta">
        {product.category} · ${product.price}
      </p>
    </div>
  </li>
);

export default ResultItem;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; }
.container { max-width: 600px; margin: 40px auto; padding: 20px; background: white; border-radius: 10px; }
h1 { text-align: center; margin-bottom: 16px; font-size: 24px; }

.search-box { display: flex; gap: 8px; }
.search-box input { flex: 1; padding: 10px 12px; font-size: 16px; border: 1px solid #ccc; border-radius:
8px; }
.search-box input:focus { outline: 2px solid #4f46e5; border-color: transparent; }
button { padding: 8px 14px; border: none; border-radius: 8px; background: #4f46e5; color: white; cursor:
pointer; font-size: 16px; }

.meta { font-size: 12px; color: #9ca3af; margin-top: 6px; }
.status { min-height: 24px; margin: 12px 0 8px; color: #4b5563; }
.error { color: #dc2626; display: flex; align-items: center; justify-content: space-between; gap: 8px;
margin-bottom: 8px; }

.results { list-style: none; border: 1px solid #e5e7eb; border-radius: 8px; transition: opacity 0.2s; }
.results.stale { opacity: 0.5; }
.result { display: flex; gap: 12px; align-items: center; padding: 10px; border-bottom: 1px solid #f3f4f6; }
.result:last-child { border-bottom: none; }
.result img { border-radius: 6px; object-fit: cover; background: #f3f4f6; }
.result-title { font-weight: 600; }
.result-meta { font-size: 13px; color: #6b7280; text-transform: capitalize; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space:
nowrap; }
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
```

```
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: request flood (debounce), minimum length and whitespace, out-of-order responses (abort), repeated queries (cache), URL encoding, HTTP errors and bad response shapes, Retry, clear, unmount mid-request, no flicker while typing.

Trade-offs to mention to the interviewer

- *Delay choice:* a shorter delay (200 ms) feels more live but sends more requests; a longer one (800 ms) feels laggy. 300-500 ms is the usual sweet spot.
- *Debounce the value (hook) vs debounce the function:* the value hook is idiomatic React and has no stale-closure problems; a debounced function suits non-React code or event handlers (it needs `useMemo` / `useRef` + cleanup in React).
- *Keep old results while loading vs clear them:* less flicker, but they must look stale (faded + `aria-busy`).
- *In-memory cache with no expiry:* fast and simple, but data can go stale and memory grows. Add a TTL / LRU limit, or use React Query / SWR in production.
- *AbortController vs an "ignore" flag:* abort also frees the network; an ignore flag only drops the result.
- *Debounce vs throttle:* debounce waits for a pause (search boxes); throttle runs at most once every N ms *while* the user keeps going (scroll, resize, live position updates).

What to add with more time

- Tests: `useDebounce` with fake timers (`vi.useFakeTimers()`); React Testing Library for "type fast → one fetch", the abort race, and error → Retry with a mocked `fetch`.
- Autocomplete dropdown with keyboard navigation (↑/↓/Enter/Esc) and `role="combobox"` / `aria-activedescendant`.
- Highlight the matched text; "Enter to search now" (flush the debounce).
- Cache with TTL / LRU; prefetch popular queries; React Query / SWR.
- Sync `?q=` to the URL; pagination / "load more" for `total > limit`.

13. Counter questions the interviewer may ask

Requirements and design

Q1. What is debouncing, in one line? How is it different from throttling?

Debounce: run the function only after the calls *stop* for N ms (only the last call counts). Throttle: run it at most once every N ms *while* the calls keep coming. Search box → debounce; scroll / resize / mousemove → throttle.

Q2. Why 400 ms? How would you choose the delay?

Typical typing gaps are 100-300 ms, so 300-500 ms catches "stopped typing" without feeling slow. I'd make it a constant, test it with real users, and lower it if the API is fast and cheap.

Q3. Why not search only when the user presses Enter?

That's simpler and sends the fewest requests, but it loses "results as you type". A good compromise: debounce + Enter flushes immediately.

Q4. Why a minimum of 2 characters?

A 1-letter query matches almost everything: a large, slow, useless response. The threshold depends on the data (for product codes, 1 may be fine), so ask.

State design

Q5. Why two values (`query` and `debouncedQuery`) instead of debouncing `setQuery`?

The input must update on every keystroke (it's a controlled input). Delaying `setQuery` makes typing lag and can drop characters. Only the side effect (the request) should be delayed.

Q6. Why is the cache a `useRef` and not `useState`?

Writing to the cache shouldn't re-render anything; the results state already does that. A ref keeps the Map across renders without causing renders.

Q7. Where would you put this state in a big app?

`query` stays local (or in the URL as `?q=`). The search results are server state: React Query / SWR handles caching, deduping, retries and cancelling for you.

Hard parts

Q8. Write a plain JavaScript `debounce` function.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

Mention `this` / `args` forwarding, and that a production version adds `cancel()` and `flush()`.

Q9. Now add `leading` (run on the first call) and `cancel`.

```
function debounce(fn, delay, { leading = false } = {}) {
  let timer;
  const debounced = function (...args) {
    const callNow = leading && !timer;
    clearTimeout(timer);
    timer = setTimeout(() => { timer = null; if (!leading) fn.apply(this, args); }, delay);
    if (callNow) fn.apply(this, args);
  };
  debounced.cancel = () => { clearTimeout(timer); timer = null; };
  return debounced;
}
```

Q10. If you use the debounced *function* in React, what goes wrong and how do you fix it?

Created in the render body, it's a new function every render (no debounce at all). With `useCallback(debounce(fn), [])` it captures stale state. Fix: keep the latest `fn` in a ref, create the debounced function once with `useMemo` / `useRef`, and `cancel()` it in an unmount cleanup.

Q11. Debounce reduces requests. Does it fix race conditions?

No. With a pause in typing, two requests can still be in flight, and the slower, older one can arrive last. You need `AbortController` (or a request id / ignore flag) so only the latest response is applied.

Q12. What does `AbortController` actually do? Does the server stop working?

The browser cancels the request and the `fetch` promise rejects with `AbortError`. The server may still finish processing it. Abort saves client bandwidth and stops stale updates; it doesn't save server CPU.

Q13. Why check `response.ok`? Doesn't `fetch` throw on errors?

`fetch` only rejects on network failure or abort. HTTP 404/500 still resolve, so without the check you'd try to read `products` from an error body and crash or show nothing.

Implementation steps

Q14. How do you prove the debounce works during the interview?

Open the Network tab and type fast: one request. I also show a request counter. For tests: fake timers. Type 5 times, advance 399 ms → no fetch; advance 1 ms → exactly one fetch with the last value.

Q15. What happens if the component unmounts while the timer or request is pending?

The `useDebounce` cleanup clears the timer; the `useProductSearch` cleanup aborts the request. So no state updates after unmount, and no leaked timers.

Q16. The user types "phone", then quickly deletes back to "ph". How many requests?

If each change is within 400 ms of the previous one, just one, for "ph": the timer restarts on every change, and only the final value is used.

Performance and scale

Q17. What if the API is slow (3 s)?

Keep the old results faded with "Searching...", allow cancelling by typing, add a timeout (`AbortSignal.timeout(8000)`) with an error + Retry, and consider showing cached or partial results first.

Q18. What if there are 1 million products?

Never search on the client. Use server-side search (a database index, Elasticsearch), return a small page (`limit=10`) plus a total, and paginate or "load more". The front end stays the same.

Q19. What if thousands of users type at the same time? How do you protect the server?

Client side: debounce + minimum length + cache + abort. Server side: rate limiting, response caching (CDN / Redis) for popular queries, and returning only the fields needed (`select=`).

Q20. Your cache grows forever. How would you fix that?

Use an LRU with a size limit (e.g. the last 50 queries) plus a TTL (e.g. 5 minutes) so data refreshes. Or use React Query's `staleTime` / `gcTime`.

Trade-offs and "why not X?"

Q21. Why not `lodash.debounce`?

In production, fine: it's tested and has `leading` / `trailing` / `maxWait` / `cancel` / `flush`. In an interview they want to see that you understand closures and timers, so write it, then mention lodash.

Q22. Why not React 18/19's `useDeferredValue` instead of debounce?

`useDeferredValue` delays *rendering* expensive UI, but it doesn't wait for a pause and doesn't reduce network calls. It's great for filtering a big local list; for API calls, use debounce.

Q23. How would you turn this into an accessible autocomplete?

`role="combobox"` on the input with `aria-expanded` + `aria-controls`, `role="listbox"` / `role="option"` on the results, `aria-activedescendant` for the highlighted option, ↑/↓/Enter/Esc handling, and close on blur / outside click.